# RAPPORT DE PROJET : SYSTÈME P2P AVEC RÉPLICATION

**Cours : Systèmes d'Information Distribués**
**Étudiante : Maguette Sow**
**Enseignant : Dr Mahamadou TOURE**

## 1. Introduction

Dans le cadre de l'unité d'enseignement sur les **Systèmes Distribués**, ce projet vise à concevoir et implémenter un système de partage de fichiers **Peer-to-Peer (P2P)**. Contrairement aux architectures client-serveur classiques, le P2P permet à chaque nœud d'agir simultanément comme client et serveur. L'objectif principal est de garantir la disponibilité des données à travers des mécanismes de **réplication active** et de **recherche distribuée**, supprimant ainsi tout point de défaillance unique (Single Point of Failure).

## 2. Architecture Technique

Le système repose sur un ensemble de technologies robustes et modernes :

- **Java 17 & Spring Boot 3.x/4.0** : Pour la rapidité de développement et la gestion native des services REST.
- **Thymeleaf** : Moteur de template utilisé pour l'interface graphique (Dashboard).
- **Maven** : Pour la gestion des dépendances et du cycle de vie du projet.
- **RestTemplate** : Utilisé pour la communication synchrone entre les nœuds via le protocole HTTP.
- **Choix du Modèle de Données (Stockage local)** : Nous avons privilégié l'utilisation du **système de fichiers local** (Filesystem) plutôt qu'une base de données centralisée ou locale. Ce choix renforce l'autonomie de chaque nœud : chaque "pair" est responsable de ses propres fichiers. Cela simplifie également le déploiement, car le nœud ne dépend d'aucun service tiers pour stocker les données binaires.

## 3. Fonctionnalités Implémentées

Le projet a été découpé en trois piliers fondamentaux :

### A. Dashboard Interactif (Interface Web)

Chaque nœud dispose d'une interface graphique accessible via /ui. Elle permet de :

- Visualiser en temps réel les fichiers stockés localement.
- Uploader un fichier via un formulaire avec un bouton "**Annuler**" dynamique en JavaScript.
- **Télécharger** ou **Supprimer** localement un document.

### B. Algorithme de Réplication et Anti-Bouclage

La réplication assure la propagation immédiate d'un fichier vers tous les pairs configurés.

- Solution au bouclage : Une vérification d'existence (Files.exists(path)) stoppe la propagation si le fichier est déjà présent, évitant ainsi des échanges infinis entre les nœuds.

### C. Résilience et Suppression Locale

La fonctionnalité de suppression a été conçue pour être locale. Cela permet de démontrer la résilience du système : même si un fichier est supprimé accidentellement ou volontairement sur un nœud, il reste disponible et téléchargeable sur les autres nœuds du réseau.

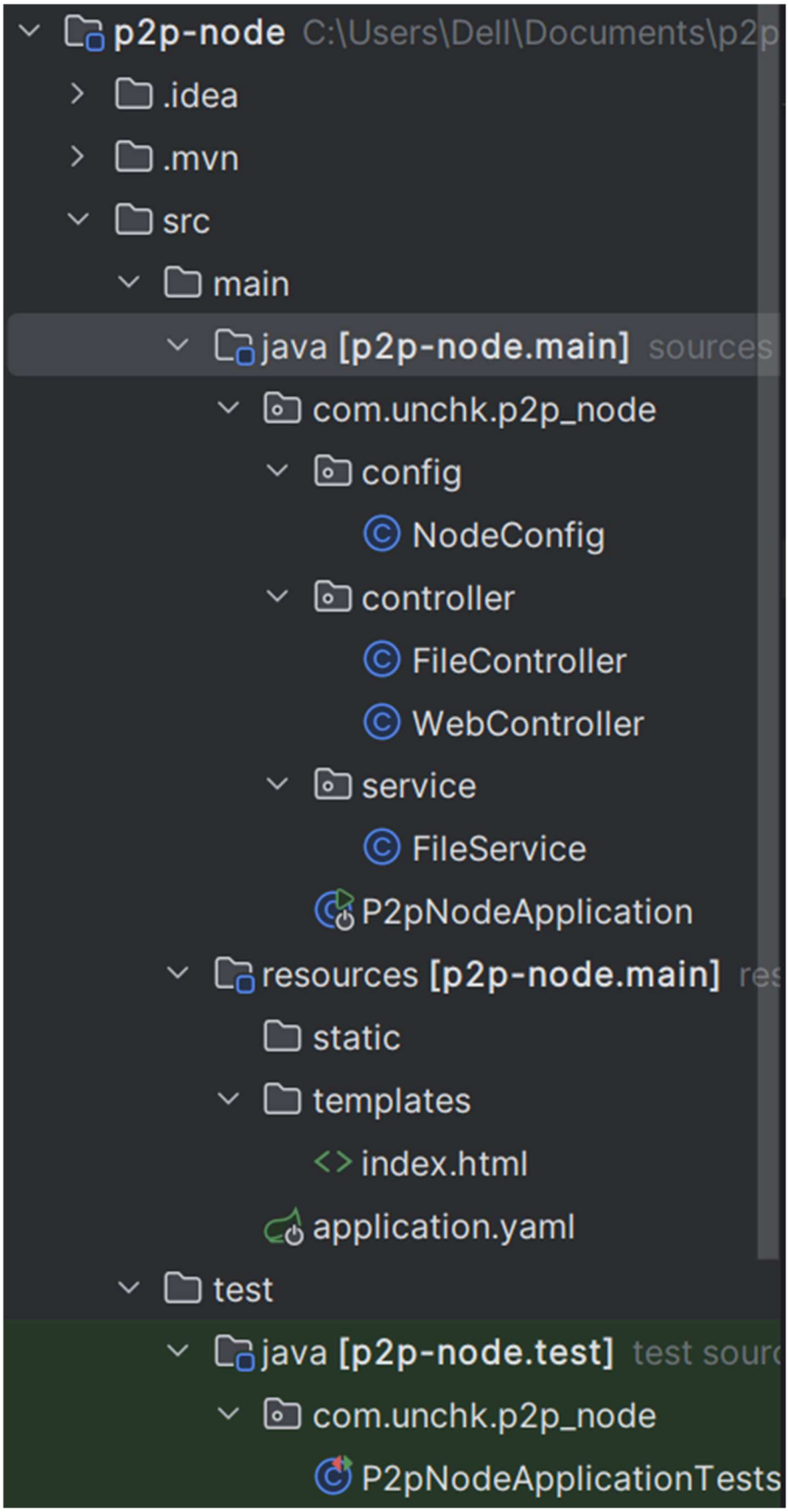
## 4. Tests et Validation

- **Test de Réplication Triple** : Upload sur le port **5000** -> Propagation automatique vers les ports **5001** et **5002**.
- **Test de Résilience** : Suppression du fichier sur le Nœud A -> Le fichier reste accessible via l'interface des Nœuds B et C.
- **Analyse des Logs** : Confirmation du flux distribué par les messages : `SUCCÈS` -> `RÉPLICATION` -> `INFO : Déjà présent`.

## 5. Captures d'Écran et Figures

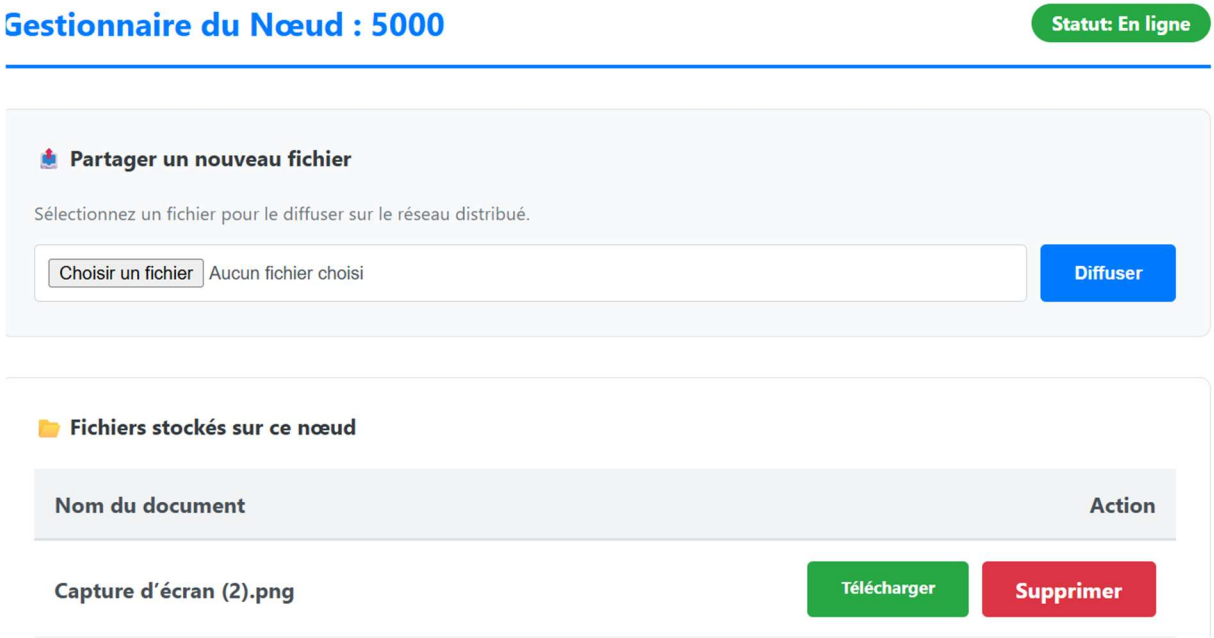
### Capture 1 : L'Arborescence du Projet

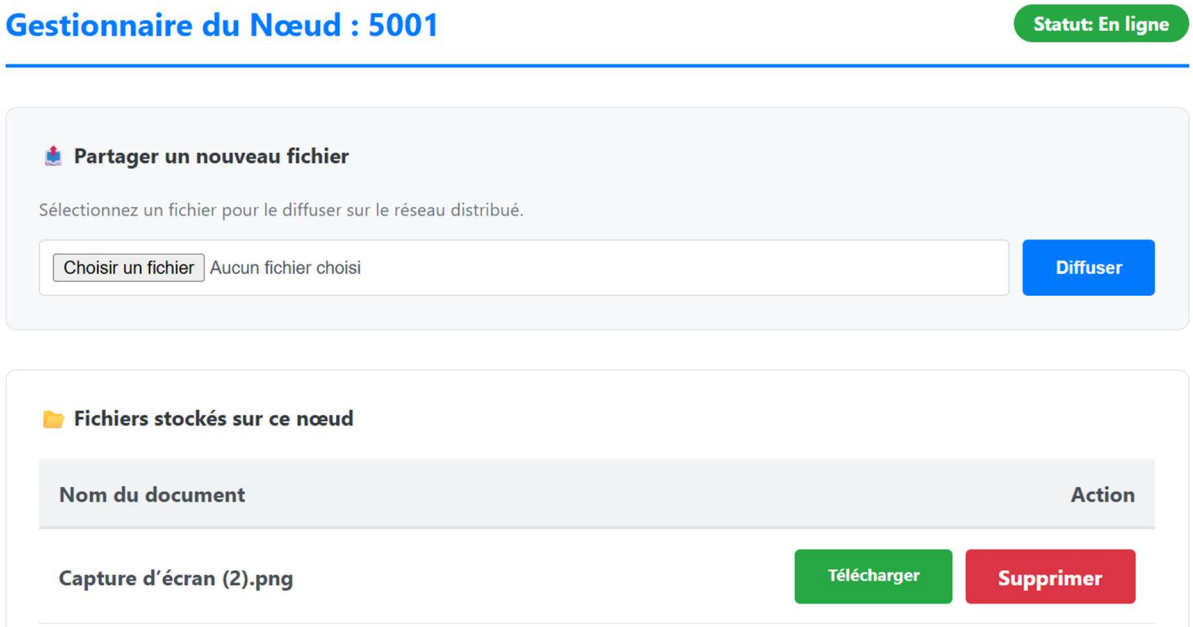
**Figure 1** : Organisation en packages (controller, service, config) et dossier templates pour l'interface Thymeleaf.

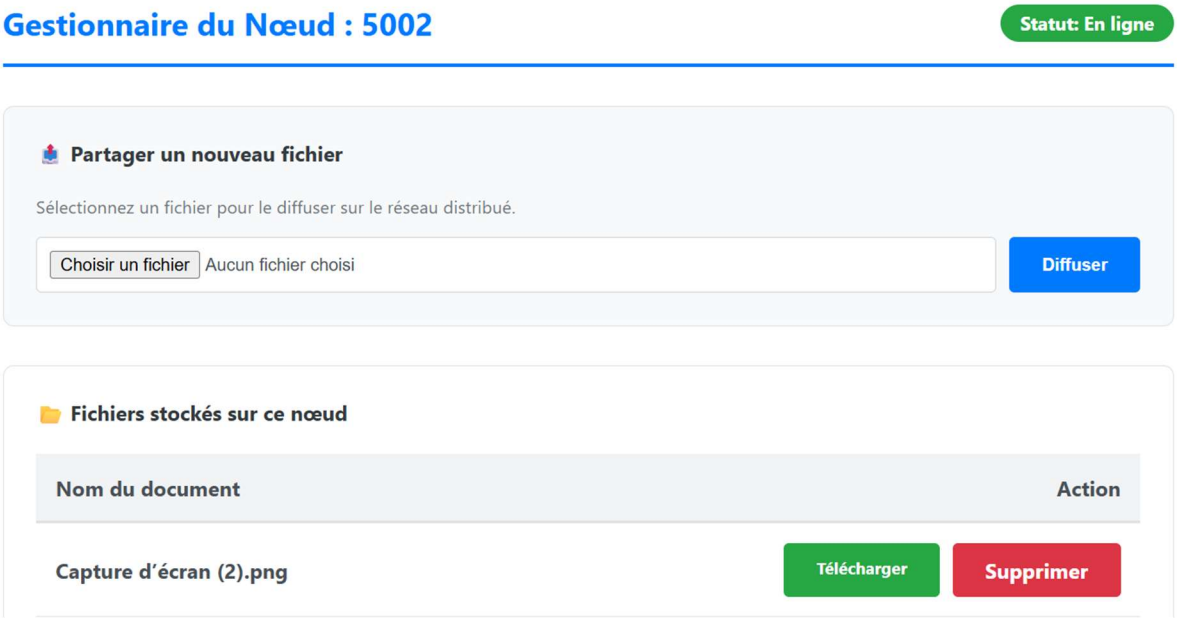

p2p-node C:\Users\Dell\Documents\p2p
.idea
.mvn
src
main
java [p2p-node.main] sources
com.unchk.p2p_node
config
NodeConfig
controller
FileController
WebController
service
FileService
P2pNodeApplication
resources [p2p-node.main] res
static
templates
index.html
application.yaml
test
java [p2p-node.test] test sourc
com.unchk.p2p_node
P2pNodeApplicationTests

**Capture 2 : Dashboard du Nœud P2P**

Figure 2 : Interface Web affichant le port du nœud, le formulaire d'upload et la liste des fichiers avec les actions Télécharger/Supprimer.






**Capture 3 : Preuve de Réplication Triple (Logs)**

Figure 3 : Logs du Nœud 5000 montrant l'envoi simultané vers deux pairs (5001 et 5002) lors d'un upload.

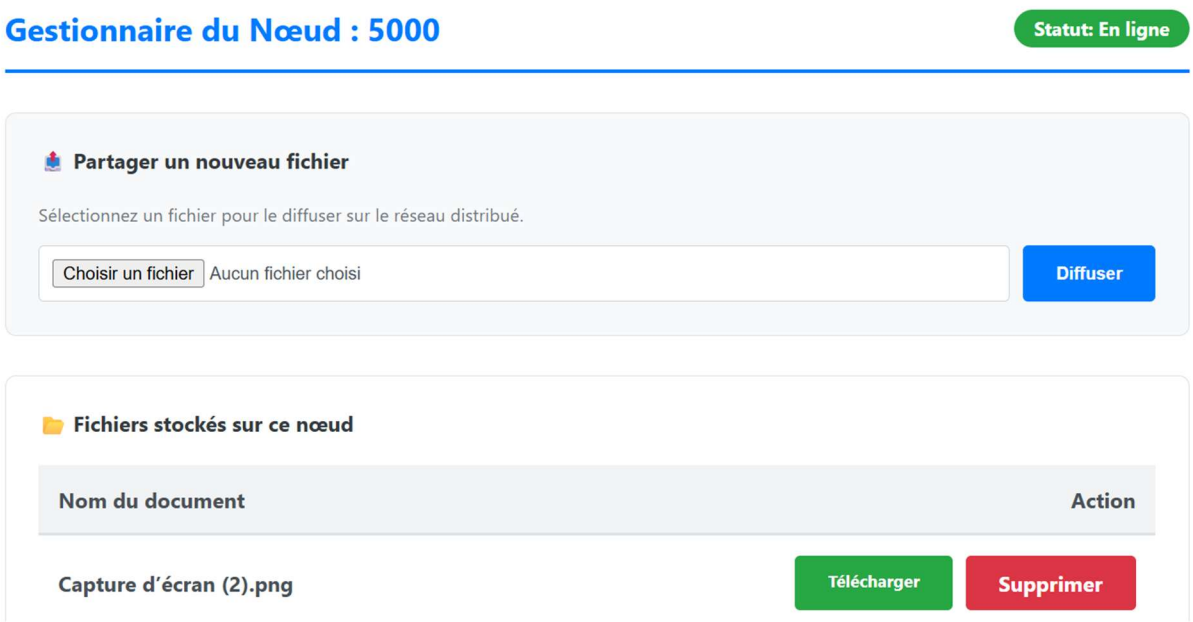
```
2026-04-21T22:35:15.823Z  INFO 8932 --- [p2p-node] [nio-5000-exec-1] o.s.web.servlet.DispatcherServlet
? SUCCÈS : Fichier [Capture d?écran (2).png] enregistré dans storage_node_5000
? RÉPLICATION : Envoi vers le pair -> http://localhost:5001
?? INFO : Fichier [Capture d?écran (2).png] déjà présent. Fin de la propagation.
? RÉPLICATION : Envoi vers le pair -> http://localhost:5002
? SUCCÈS : Fichier [Capture d?écran (3).png] enregistré dans storage_node_5000
? RÉPLICATION : Envoi vers le pair -> http://localhost:5001
? RÉPLICATION : Envoi vers le pair -> http://localhost:5002
?? INFO : Fichier [Capture d?écran (3).png] déjà présent. Fin de la propagation.
```

```
2026-04-21T22:35:27.227Z  INFO 10680 --- [p2p-node] [nio-5001-exec-2] o.s.web.servlet
ed initialization in 2 ms
? SUCCÈS : Fichier [Capture d?écran (2).png] enregistré dans storage_node_5001
? RÉPLICATION : Envoi vers le pair -> http://localhost:5000
?? INFO : Fichier [Capture d?écran (2).png] déjà présent. Fin de la propagation.
? SUCCÈS : Fichier [Capture d?écran (3).png] enregistré dans storage_node_5001
? RÉPLICATION : Envoi vers le pair -> http://localhost:5000
?? INFO : Fichier [Capture d?écran (3).png] déjà présent. Fin de la propagation.
?? INFO : Fichier [Capture d?écran (3).png] déjà présent. Fin de la propagation.
```

```
2026-04-21T22:35:48.126Z  INFO 20584 --- [p2p-node] [nio-5002-exec-1] o.s.web.servlet.
ed initialization in 2 ms
? SUCCÈS : Fichier [Capture d?écran (2).png] enregistré dans storage_node_5002
? RÉPLICATION : Envoi vers le pair -> http://localhost:5000
?? INFO : Fichier [Capture d?écran (2).png] déjà présent. Fin de la propagation.
? RÉPLICATION : Envoi vers le pair -> http://localhost:5001
? SUCCÈS : Fichier [Capture d?écran (3).png] enregistré dans storage_node_5002
? RÉPLICATION : Envoi vers le pair -> http://localhost:5000
? RÉPLICATION : Envoi vers le pair -> http://localhost:5001
```

**Capture 4 : Test de Suppression et Résilience**

Figure 4 : État du réseau après suppression locale sur le port 5000. On observe que les dossiers storage_node_5001 et 5002 conservent l'intégrité de la donnée.



## Gestionnaire du Nœud : 5001

Statut: En ligne

### Partager un nouveau fichier

Sélectionnez un fichier pour le diffuser sur le réseau distribué.

Choisir un fichier Aucun fichier choisi

Diffuser

### Fichiers stockés sur ce nœud

| Nom du document | Action |
|---|---|
| Capture d'écran (2).png | Télécharger Supprimer |
| Capture d'écran (3).png | Télécharger Supprimer |

## Gestionnaire du Nœud : 5002

Statut: En ligne

### Partager un nouveau fichier

Sélectionnez un fichier pour le diffuser sur le réseau distribué.

Choisir un fichier Aucun fichier choisi

Diffuser

### Fichiers stockés sur ce nœud

| Nom du document | Action |
|---|---|
| Capture d'écran (2).png | Télécharger Supprimer |
| Capture d'écran (3).png | Télécharger Supprimer |

## 6. Conclusion

Ce projet démontre la viabilité d'une architecture décentralisée pour le stockage de données. Le système est résilient : la perte d'un nœud n'entraîne pas la perte de l'information.

**Améliorations futures :**

1. **Sécurité** : Implémenter le protocole **HTTPS (TLS/SSL)** pour sécuriser les transferts entre pairs.
2. **Scalabilité** : Utiliser une **Table de Hachage Distribuée (DHT)** pour gérer des milliers de nœuds sans saturer le réseau par des messages de recherche.
3. **Élection de Leader** : Ajouter un algorithme d'élection pour désigner un nœud coordinateur en cas de conflit de version.